Advanced Computer Vision

Assignment #1

Soil Grain-Size Prediction with CNNs, ViTs and Self-Supervised Models

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 17 points*
- *Timeline: Assignment #1 available on 13th Oct, Code submission deadline till 30th Oct, and offline defense deadline till 15th Nov*

Take part in a Kaggle competition: from smartphone photos of a soil sample, predict its grain-size distribution, i.e. the percentage of soil (by mass) passing each of 11 sieves (0.002–200 mm). The data set is tiny (a few dozen physical samples, several photos each), the photos are huge (12–48 MP), each photo comes with a physical scale (pixels per mm), and the domain is far from ImageNet. Grain size is a physical length: any resize or crop that changes millimetres-per-pixel changes the correct answer.

You will solve the task three times: with CNNs, with Vision Transformers and by tuning self-supervised models, and learn from first-hand experience how the optimisation of these families differs, what each of them looks at, and how to combine them.

You may use AI tools for code (see the AI Tools usage policy). The grade therefore depends mostly on what code generation cannot do for you: hypotheses about this particular data set, experiments that test them, and your ability to explain the behaviour of your models at the defense.

While completing this assignment, you will develop and strengthen the following skills:

- *Building and tuning CNN, ViT and SSL-based pipelines on the same data and validation, and explaining why their optimisation recipes differ.*
- *Designing a group-aware validation strategy for a tiny data set and keeping CV and leaderboard consistent.*
- *Handling very large images with a physically calibrated input pipeline.*
- *Interpreting models with Grad-CAM and attention maps and using them to detect shortcut learning.*
- *Comparing model families by quality, cost and robustness, and building a useful ensemble.*

- *Formulating a hypothesis, predicting an experimental outcome in advance, and learning from a wrong prediction.*

Assignment deliverables (what students submit):

- *A reproducible repository with: tiling / caching pipeline and the validation folds file; training and inference code for every approach; config.yaml (or equivalent) with reproducible settings for every reported experiment; a lockfile (uv.lock or poetry.lock) and pyproject.toml; an inference script (photo(s) + scale → valid grain-size curve).*
- ***A results table (mandatory): CV score (mean ± std across folds) and Public / Private LB score for every experiment and every submission.***
- *Artifacts: out-of-fold (OOF) predictions for every experiment in the results table, the final submission, links to reported checkpoints, a compute log (GPU type, hours, peak memory per experiment).*
- *A hypotheses log (HYPOTHESES.md): before each block of experiments, the hypothesis and the predicted outcome, committed before the corresponding results (the git history is checked); after the block, what happened and why.*

Task

Task Selection:

- *Take part in the Kaggle competition [Predicting Soil Grain Size Distributions from Images]*

Part 0 – EDA, Metric and Validation:

- *Perform a short Exploratory Data Analysis (EDA): samples, photos per sample, devices, scale range, curve shapes (fine vs coarse soils).*
- *Implement the competition metric. Analyze its strengths, weaknesses and edge cases (e.g. what a 2× scale error costs).*
- *Suggest a complementary metric (e.g. D10 / D50 / D60 error) and propose an alternative target metric, with justification.*
- *Design a group-aware validation: photos of one physical sample are never in both train and validation; predictions are aggregated to one curve per sample before scoring. Use the same folds for all experiments.*
- *Build the input pipeline: tiles of a fixed physical size (in mm), cached once; aggregation of tile predictions to a photo / sample prediction. Justify the tile size.*

Part 1 – CNN Approach:

- *Start with a simple pretrained CNN (e.g. ResNet), then move to stronger architectures (e.g. ConvNeXt / EfficientNet).*
- *Explore several optimisation setups: SGD vs AdamW, LR schedule and warmup, BatchNorm with small batches (GroupNorm, frozen BN statistics), augmentations, input resolution.*
- *Answer: which inductive biases of CNNs help for soil texture and which hurt? What happens to BatchNorm when a batch contains tiles of only a few physical samples?*
- *Success criteria: table architectures × optimisation setups with CV ± std, LB and GPU-hours; hypotheses written before the runs, and an explanation of the observed differences.*
- ***Prepare an ablation study on the tested models and methods***

Part 2 – ViT Approach:

- *Start with your own ViT (the Lecture 1 implementation) trained from scratch, then move to pretrained architectures (e.g. ViT / DeiT → Swin / MaxViT).*

- *Explore several optimisation setups: warmup, layer-wise LR decay, weight decay (and which parameters it is applied to), patch size and resolution, stochastic depth, augmentations.*
- *Predict before running how your from-scratch ViT will compare with a pretrained ViT and with your pretrained CNN on this data, then explain the result. Why does pretraining matter more for ViTs than for CNNs?*
- *Success criteria: table architectures × optimisation setups with CV ± std, LB and GPU-hours; a comparison of the optimisation recipes that worked for CNNs and for ViTs, with an explanation why they differ.*
- ***Prepare an ablation study on the tested models and methods***

Part 3 – Tuning Self-Supervised Models:

- *Use at least two self-supervised encoders (e.g. DINOv2, DINOv3, MAE). Go through: frozen features (k-NN / ridge) → linear probe → LP-FT → full fine-tuning or LoRA.*
- *Domain adaptation: continue self-supervised pre-training (DINO-style) of one encoder on unlabelled tiles of the training photos, then repeat the comparison.*
- *Answer: which augmentations are label-destroying for this task and why (hint: scale)? Can fine-tuning be worse than frozen features here, and if so, why? Support the answer with CKA between the encoder before and after tuning.*
- *Success criteria: table encoders × tuning strategies (frozen / LP / LP-FT / FT or LoRA), before and after domain adaptation, on the same folds; training curves of the domain-adaptation run, and a clear answer whether it helped, by how much and at what GPU cost.*
- ***Prepare an ablation study on the tested models and methods***

Part 4 – Interpretation:

- *Grad-CAM (or a similar method) for CNNs; attention maps / attention rollout for ViTs and SSL models.*
- *The goal is not pretty maps but a test for shortcuts: does the model look at the grains, or at background, lighting, rulers, coins, shadows? Design at least one control experiment that confirms or refutes a shortcut (e.g. masked background, colour-only input, shuffled patches, prediction from scale / EXIF only).*
- *Success criteria: maps for the same photos across all three approaches, including failure cases; at least one control experiment with a numeric result and a conclusion.*

Part 5 – Conclusions on Optimisation:

- *For each approach: which optimisation strategies worked, which did not, and why.*
- *Discuss explicitly the hypotheses from your log that turned out wrong.*

Part 6 – Comparison of Approaches:

- *Strengths and weaknesses of CNN, ViT and SSL tuning: CV / LB, GPU-hours, memory, data efficiency and robustness to scale (the same photo at a different scale with the corresponding pixels-per-mm).*
- *On which samples (fine vs coarse soils, devices) does each approach fail?*
- *Success criteria: one comparison table and a scale-robustness plot for the best model of each approach.*

Part 7 – Ensemble:

- *Measure the correlation of OOF errors between approaches; build the ensemble from diverse models, not only from the best ones.*
- *Show how much the ensemble gains over the best single model and explain why.*

- *Choose the final submission by CV, not by the public LB.*

Use small and medium models (ResNet-18/50, ConvNeXt-T, ViT-S, Swin-T, MaxViT-T), mixed precision (fp16 on T4 / P100), cached tiles and cached frozen features.

Final Report

- *Prepare a comprehensive report including:*
  - *data insights,*
  - *feature engineering,*
  - *tried algorithms and models,*
  - *chosen hyperparameters,*
  - *metrics,*
  - *insights and conclusions.*
- *Reports may be prepared in LaTeX or Markdown (README.md).*

NOTES

- *Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a "lucky shot." Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself, and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.*
- *Keep your EDA concise. Avoid overloading it with excessive plots, histograms, or tables. Focus only on key aspects of the data (e.g., missing values, target distribution). If a part of your EDA does not lead to meaningful conclusions, you likely don't need it (for example, full pairwise correlation matrices). Do NOT follow the typical Kaggle-style EDA — most of them are far too verbose and unfocused.*
- *Prepare your presentation carefully. A messy speech with too much code or too many files makes it nearly impossible for the audience to follow. Think in advance about what you want to highlight*
- *DO NOT USE ANY ADDITIONAL LABELLED SOIL DATA AND DO NOT LABEL DATA MANUALLY — this is a RED FLAG and will result in an instant BAN! Publicly available pretrained weights are allowed and must be listed with exact names / versions.*
- *Unlabelled test photos may be used for self-supervised pre-training or pseudo-labelling only if the competition rules allow it; otherwise use training photos only. State explicitly what you used.*
- *DO NOT hard-code anything that depends on specific test image ids or file names — RED FLAG, instant BAN!*
- *Use the same splits for all approaches so that the results can be compared. Your final report MUST include both CV and LB scores for all experiments. The public LB is a small part of a tiny test set — do not select models on it.*
- *A wrong prediction in the hypotheses log with a good explanation is graded as well as a correct one.*
- *Instrument your code from day one: every experiment writes its config, OOF predictions, metrics and compute usage to disk.*

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	***Points***
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	*5*
Practical question about the submitted code during defense	*7*
Theoretical part during defense (2 questions + 1 exercise)	*5*
Criterion	***Points***

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 1, but will not necessarily be restricted to them.

Exercise with practical tasks similar to Appendix 2 will be evaluated in the form of a written test on one of the practical sessions

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- *Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.*
- *Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.*
- *Language assistance tools: Grammarly, DeepL, Google Translate, etc.*
- *Local LLMs or self-hosted AI models.*

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- *Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.*
- *Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.*

- *Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.*
- *Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.*
- *Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.*
- *Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.*

Conditions for Use

Full Transparency:

- *Any use of AI tools must be disclosed in your submission.*
- *Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.*
- *If using online tools, include chat links (if available) or full transcripts in an appendix of your report.*

Partial Contribution Only:

- *AI tools should support your work — they must not produce the complete assignment, lab, or report.*
- *You must be able to explain and defend any AI-generated content during oral defense or Q&A*

Examples

Acceptable Uses:

- *"Correct grammar and style in this paragraph of my report."*
- *"Write a function to load images from a directory and resize them."*
- *"Suggest 3 ideas for improving model generalization."*

Unacceptable Uses:

- *"Write the entire peer review for this paper."*
- *"Generate the complete solution code for this assignment task."*

Appendix 1 (Theoretical Questions)

Vision Transformers and resolution

- *Estimate attention cost and memory for a 1024 × 1024 input with patch size 16 and compare with tiling. When is global context worth the quadratic cost?*
- *Prove that self-attention without positional information is permutation-equivariant and that mean pooling then makes the model permutation-invariant. What does this imply for texture-like inputs such as soil?*
- *Compare learned absolute positional embeddings, 2D sin-cos, relative position bias and RoPE with respect to changing the input resolution.*

- *What does PI-resize in FlexiViT preserve, and why is naive bilinear resizing of the patch-embedding kernel wrong?*
- *What problems do LayerScale, QK-norm and register tokens solve? Which of them matter for dense / texture features?*
- *Why are FLOPs not latency? How would you measure throughput fairly on a T4?*

CNN vs ViT optimisation

- *Why do ViTs usually need AdamW, warmup and strong augmentation / regularisation, while CNNs train well with SGD + momentum? Relate it to inductive biases and to the loss landscape.*
- *What goes wrong with BatchNorm when the batch is small or contains highly correlated samples? Compare GroupNorm, LayerNorm and frozen BN statistics for fine-tuning.*
- *What is layer-wise LR decay and why does it help fine-tuning pretrained ViTs? Why are biases and normalisation parameters usually excluded from weight decay?*
- *Swin and MaxViT vs plain ViT: what do local / window attention and convolutional stems change for small data sets and large images?*
- *Why is a ViT trained from scratch data-hungry compared with a CNN? Which inductive biases does a CNN have?*

Representation evaluation and transfer

- *What does weighted k-NN on frozen features measure? Why L2-normalise and use a temperature?*
- *Describe a leakage-free linear-probe protocol for a grouped data set with a few dozen groups.*
- *Why can intermediate layers be better than the last one for a texture-statistics task?*
- *Give an example where frozen-feature rankings disagree with fine-tuning rankings (MAE). Why can this happen?*
- *Explain feature distortion and why LP-FT reduces it. Relate it to the state of the head at the start of fine-tuning.*
- *Define linear CKA and RankMe. What can each of them tell you, and what not, about downstream quality?*
- *LoRA: write the update, count its parameters and explain why it adds no inference cost after merging. When is LoRA a worse choice than partial fine-tuning?*

Self-supervised learning

- *Contrastive vs self-distillation vs masked modelling: what is the learning signal of each, and which inductive biases do the augmentations inject?*
- *How does DINO avoid collapse without negative pairs? Explain the separate roles of centering and sharpening and what happens if either is removed.*
- *Why is the teacher an exponential moving average of the student? What happens with momentum 0 or 1?*
- *What does the iBOT masked-patch loss add on top of DINO, and why does it matter for dense features?*
- *What are KoLeo regularisation (DINOv2) and Gram anchoring (DINOv3), and which failure mode does each address?*
- *Why can continued in-domain SSL help, and how can it hurt (forgetting, augmentation mismatch, small data)? How would you detect each?*
- *Which augmentations are label-destroying for grain-size prediction, and how would you design a scale-safe multi-crop?*

Metric, validation, interpretation and ensembles

- *Show that for distributions on an ordered 1-D grid the earth mover's distance equals the L1 distance between cumulative distributions. Why does the log-diameter axis matter?*
- *Why is MSE on the curve a poor surrogate for EMD in the tails of the distribution?*
- *Why is group-aware validation essential here, and what does leave-one-sample-out cost in variance?*
- *How is Grad-CAM computed? Why is it coarse for CNNs, and why is it not directly applicable to ViTs?*
- *Attention maps vs attention rollout: what does each show, and why can attention maps be misleading as explanations?*
- *What is shortcut learning? Give two possible shortcuts in this competition and an experiment that would reveal each.*
- *Why can an ensemble of diverse models beat an ensemble of the best models? How do you measure diversity on OOF predictions?*
- *With a limited GPU budget, how would you decide between more SSL steps, a bigger backbone and more fine-tuning? What evidence would you collect first?*

Appendix 2 (Practical Exercises)

Exercise 1. Tokens and compute for large photos

ViT-S/16: $D = 384$, 6 heads, 12 blocks. A 1024×1024 crop is fed to the model.

1) Number of tokens N (including [CLS]).

2) fp16 memory of one attention matrix (one head, one layer).

3) MACs of one block, $12ND^2 + 2N^2D$. Which term dominates?

4) Total MACs for the 1024^2 crop vs. 16 non-overlapping 224×224 tiles.

Exercise 2. Swin vs. global attention

Stage 1 of Swin-T: $h \times w = 56 \times 56$ tokens, $C = 96$, window $M = 7$.

Compute $\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$ and $\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$, and their ratio.

Exercise 3. Weighted k-NN

A test embedding has 4 neighbours with cosine similarities $s = (0.9,\ 0.8,\ 0.75,\ 0.6)$ and classes (A, B, B, B). Neighbour weights are $w_i = \exp(s_i/\tau)$.

1) Predicted class for $\tau = 0.1$, for $\tau = 1$, and for plain majority vote.

2) Regression: the neighbours' percent passing at one sieve is (60, 40, 50, 30).

Predict it with $\tau = 0.1$.

Exercise 4. Effective rank

Embeddings have singular values $\sigma = (4, 2, 1, 1)$. Compute

$$\text{RankMe} = \exp\left(-\sum_k p_k \log p_k\right), \quad p_k = \sigma_k / \sum_j \sigma_j.$$

What are the maximum and minimum possible values for $D = 4$?

Exercise 5. Linear CKA

$$X = [1, 2, 3, 4]^\top, \quad Y_1 = [2, 4, 6, 8]^\top, \quad Y_2 = [1, -1, 1, -1]^\top.$$

On centred features compute $\text{CKA}(X, Y_1)$ and $\text{CKA}(X, Y_2)$ with

$$\text{CKA}(X, Y) = \|Y^\top X\|_F^2 \,/\, \left(\|X^\top X\|_F \, \|Y^\top Y\|_F\right).$$

Exercise 6. Making a valid curve

A model predicts percent passing at 5 increasing sieves: $\hat{y} = (10, 35, 30, 80, 105)$.

A valid curve is non-decreasing, lies in [0, 100] and ends at 100.

1) Fix $\hat{y}$ with clipping + running maximum.

2) Fix $\hat{y}$ with clipping + isotonic regression (pool-adjacent-violators).

3) Which fix is closer in squared error to the clipped prediction, and why?

Exercise 7. EMD on a log grid

Sieves $d = (0.1, 1, 10)$ mm. Predicted percent passing $P = (20, 50, 100)$, true $T = (40, 60, 100)$.

$\mathrm{EMD} = \sum_{i=1}^{K-1} |P_i - T_i| \cdot (\log_{10} d_{i+1} - \log_{10} d_i)$.

1) Compute the EMD.

2) Compute D_{50} (diameter at 50% passing, linear interpolation in $\log_{10} d$) for both curves.

3) The photo was processed at a wrong scale: every grain looks $2\times$ smaller and the predicted curve shifts as a whole along $\log d$. Assuming both curves stay inside the sieve range, what EMD does this error alone cause?

Exercise 8. DINO centering and sharpening

Teacher logits for one image $t = (2, 0, 0)$, centre $c = (1, 0, -1)$, teacher temperature $\tau_t = 0.5$.

1) Teacher probabilities $\mathrm{softmax}((t - c)/\tau_t)$.

2) The same without centering.

3) Centre update $c \leftarrow m\,c + (1 - m)\,\bar{t}$ with $m = 0.9$, where the batch mean $\bar{t} = t$.

4) Which mechanism prevents collapse to one dimension, and which prevents collapse to the uniform distribution?

Exercise 9. LoRA parameter count

ViT-S: $D = 384$, 12 blocks, $\approx$ 22M parameters. LoRA with rank $r = 8$, $\Delta W = BA$, is applied to the fused qkv projection ($384 \to 1152$) and to the attention output projection ($384 \to 384$) in every block.

1) Number of trainable LoRA parameters.

2) Their share of the backbone.

3) Number of parameters in one full qkv weight matrix, for comparison.

Exercise 10. Grad-CAM by hand

The last convolutional layer has 2 channels with 2×2 activation maps

A^1: rows (1, 0) and (2, 1); A^2: rows (0, 3) and (1, 0).

The gradients of the output are $\partial y/\partial A^1_{ij} = 0.5$ and $\partial y/\partial A^2_{ij} = -0.25$ for all i, j.

1) Channel weights $\alpha_k = \frac{1}{Z}\sum_{i,j} \partial y/\partial A^k_{ij}$.

2) The Grad-CAM map $L = \mathrm{ReLU}\left(\sum_k \alpha_k A^k\right)$.

3) Which location is the most important, and why is the ReLU applied?